

Software Architecture and Design Specification (SAD)

Hostel Management System

Project: Hostel Management System

Team: Team 5

Version: 1.0

Date: 03-10-2026

Status: Draft for Review

Team Members

Name	SRN
Arjav Chandranath	PES2UG24CS080
Arjun Mohan Gowda	PES2UG24CS082
Armaan Ali	PES2UG24CS083
Arnav Sharath Hanchanur	PES2UG24CS084

1. Introduction

1.1 Purpose

This document explains how the Hostel Management System (HMS) is built: its main parts, how they work together, why each technology was chosen, and how the important flows and interfaces are designed in detail. It turns the requirements in the Software Requirements Specification (SRS v1.0) into a design that developers can program from and that testers can check against the Software Test Plan (STP v1.0).

1.2 Scope

The design covers every requirement in SRS v1.0: 20 functional requirements, 6 non-functional requirements and 5 security requirements. In plain terms, it covers:

- Registration, login, profiles, password reset and role-based access for Students, Wardens/Admins and the optional Technician role.
- Hostel block, floor and room inventory, room allocation, reallocation and room-change requests.
- Fee structures, fee statements, online payment through a sandbox payment gateway, PDF receipts, reconciliation and defaulter lists.
- Maintenance complaints with photo attachments and a tracked status workflow.
- Occupancy, vacancy and fee-collection reports with CSV and PDF export.

Out of scope, as stated in SRS Section 1.2: biometric access, CCTV, mess/canteen management, and any integration with the university ERP beyond checking a student's roll number/ID. The optional REST API (Django REST Framework) and the Technician sub-role are designed here but are built only if they ship in Sprint 3.

1.3 Audience

Developers and testers on Team 5, the Course Coordinator and evaluators, security reviewers, and future student teams who will maintain the system.

1.4 Definitions

HMS (Hostel Management System), SRS (Software Requirements Specification), STP (Software Test Plan), RTM (Requirements Traceability Matrix), SAD (this document), ADR (Architecture Decision Record), RBAC (Role-Based Access Control), OTP (One-Time Password), TLS (Transport Layer Security), PCI-DSS (Payment Card Industry Data Security Standard), ORM (Object-Relational Mapping), DRF (Django REST Framework), CSRF (Cross-Site Request Forgery), STRIDE (a threat-classification method). Plain-language explanations are in Appendix 5.1.

2. Document Overview

2.1 How to use this document

This document provides the architecture and design deliverables for HMS: a component (UML) diagram, architecture decision records, a threat model, three UML sequence diagrams, and the API design. Use the table below to find what you need.

If you want to...	Read
See the big picture of the system	3.3 Component diagram, 3.4 Component descriptions
Understand why the system is built this way	3.5 Pattern and decisions (ADRs), 3.6 Technology stack
Find which design element satisfies a requirement	3.8 Traceability to requirements (DS-* references)
Review security	3.9 Security architecture, 4.4 Error handling and logging
Implement or test a flow	4.2 Sequence diagrams, 4.3 API design
See what is still undecided	4.6 Open issues and next steps

2.2 Related Documents

Document	Version / Date	How it relates to this SAD
Software Requirements Specification (SRS)	v1.0, 04-09-2026	Source of all requirement IDs (HMS-F-###, HMS-NF-###, HMS-SR-###) that this design satisfies.
Software Test Plan (STP)	v1.0, 16-09-2026	Defines the tests (TC-*) that check this design. Its RTM (Section 13.2) already uses the DS-* references defined here.
Requirements Traceability Matrix (RTM)	SRS Section 8; STP Section 13.2	Links requirement, design reference, module and test case. Section 3.8 below completes the design column.
Jackfruit Project Guidelines, UE24CS341A	2026	Course rules on process, tools and deliverables.

3. Architecture

3.1 Goals & Constraints

Goals:

The architecture is meant to be secure, correct, easy to maintain, and simple enough for a four-person team to finish in one semester. The measurable targets come from the SRS.

Goal	Target	Source
Security	Only the right role can see or change data; passwords hashed; no card data stored; TLS 1.2+ everywhere	HMS-SR-001 to SR-005, HMS-NF-003
Data correctness	A room can never hold more students than its capacity, even when two wardens act at the same time	HMS-F-012, HMS-F-015
Performance	95% of page loads and form submissions finish within 2 seconds for up to 200 concurrent users	HMS-NF-001
Reliability	99.5% availability during the semester, excluding planned maintenance	HMS-NF-002
Auditability	Every admin action and fee transaction is recorded and kept for at least 3 years	HMS-SR-004, HMS-NF-004
Usability	Works from 360 px wide screens; follows WCAG 2.1 AA where practical	HMS-NF-005
Scalability and maintainability	Supports 5,000 students and 2,000 rooms without redesign; one Django app per feature	HMS-NF-006

Constraints:

- Must be finished and demonstrable within one academic semester by a team of four.
- Payments use only the sandbox/test mode of a PCI-DSS compliant gateway (Razorpay or Stripe). HMS must never handle real card data.
- Relational storage (PostgreSQL) is required to meet the course's DBMS expectations. SQLite is not used, so that database constraints are really exercised in tests.
- Backend is Python/Django with a server-rendered front end (SRS 2.1). Server runs on Linux behind Nginx and Gunicorn.
- Integration with the university is limited to checking a roll number/ID during registration.
- Email/SMS delivery and card capture are third-party services; HMS controls only its side of each interface.

3.2 Stakeholders & Concerns

Stakeholder	Main concerns	How the architecture responds
Students	Simple, mobile-friendly pages; safe payment; privacy of personal data; clear status of complaints	Responsive server-rendered UI; gateway-hosted checkout; per-user data checks (no access to other students' records); status history on complaints

Stakeholder	Main concerns	How the architecture responds
Warden / Admin	Fast bulk work; no double-booked rooms; clear audit trail; accurate reports	Atomic allocation with row locking and a database capacity rule; append-only audit log; reports computed from the same tables as transactions
Technician (optional)	See only the complaints assigned to them	Dedicated role with access limited to ComplaintModule views
Finance / hostel office	Payments match the gateway ledger; defaulters are correct	Webhook-driven payment confirmation, idempotent updates, reconciliation against the gateway ledger
Course Coordinator and evaluators	Traceability from requirements to design to tests; sound engineering practice	DS-* references in 3.8; ADRs in 3.5; tests mapped in the STP
Developers and maintainers	Modularity, readability, easy testing	One Django app per feature; service layer separate from views; adapters with mock versions for external services
Security reviewers	Access control, data protection, injection and session safety	Security architecture in 3.9 mapped to SR-001 to SR-005 and the proposed SR-006 onwards

3.3 Component (UML) Diagram

Figure 1 shows the main components of HMS and how they connect. Solid arrows are calls. The dashed arrow from the payment gateway is the signed webhook coming into HMS. Dotted arrows are file storage. Blue boxes are HMS components, green boxes are adapters that hide external services, and grey boxes are external services. The dashed REST API box is optional.

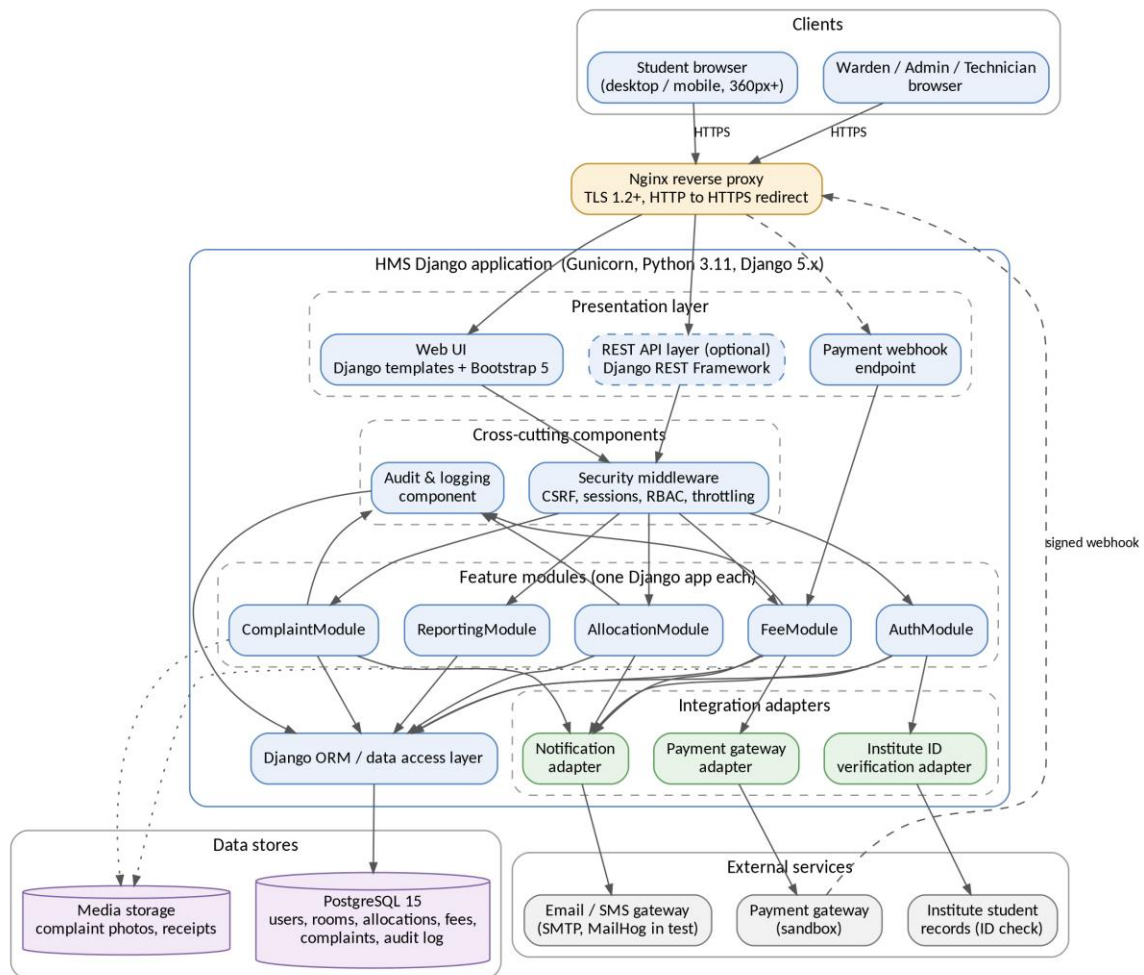


Figure 1: HMS component diagram

3.4 Component Descriptions

Component	Responsibility	Requirements	Depends on
Nginx reverse proxy	Ends TLS, redirects HTTP to HTTPS, sets HSTS, serves static files, limits request size, forwards to Unicorn	SR-001	TLS certificate
Web UI	Server-rendered Django templates with Bootstrap 5: Student dashboard and Admin/Warden dashboard. Browser-side checks are only hints; every rule is checked again on the server	NF-005, all user-facing FRs	Security middleware, feature modules
REST API layer (optional)	JSON endpoints for allocation, fee and complaint data, built with Django REST Framework. Calls the same service functions as the UI. Uses JWT only if exposed outside the web app	SR-003, FRs via API	Security middleware, feature modules
Payment webhook endpoint	Receives signed callbacks from the gateway. Exempt from CSRF but rejects any call whose signature does not verify	F-021, F-024, SR-005	FeeModule, payment gateway adapter
Security middleware	Session handling (20-minute idle timeout), CSRF protection, role checks on every view and API endpoint, login and OTP throttling	F-004, F-005, SR-003	AuthModule, PostgreSQL
AuthModule	Registration with roll-number check, login/logout, profile update, OTP password reset, role assignment, password hashing	F-001 to F-005, SR-002	ID verification adapter, notification adapter
AllocationModule	Blocks, floors, rooms and capacity; student record CRUD; allocate and reallocate; room-change requests; capacity and gender-policy rules	F-010 to F-015	Audit component, notification adapter
FeeModule	Fee structures per term and room type; itemised statements; payment start; webhook handling; PDF receipts; reconciliation and defaulter list	F-020 to F-024, SR-005	Payment gateway adapter, audit, notification adapter
ComplaintModule	Complaint tickets with category, description and optional photo; status workflow Open, In Progress, Resolved; student notification	F-030, F-031	Media storage, audit, notification adapter
ReportingModule	Read-only occupancy, vacancy, fee-collection and defaulter reports for a date range; CSV and PDF export	F-040, F-041	Data access layer
Audit & logging component	Append-only audit table for admin actions and fee transactions; structured application logs without sensitive data	SR-004, NF-004	PostgreSQL
Payment gateway adapter	One small interface (create order, verify signature, fetch ledger) with a real sandbox version and a mock version for offline tests	F-021, SR-005	Payment gateway (sandbox)
Notification adapter	Sends OTPs, allocation notices, payment confirmations, fee reminders and complaint updates by email/SMS; MailHog catches mail in test	F-003, F-021, F-031	Email/SMS gateway
Institute ID verification adapter	Checks that a roll number/ID is valid and not already registered	F-001	Institute student records

Component	Responsibility	Requirements	Depends on
Data access layer (Django ORM)	Models, migrations, queries and transactions; builds parameterised SQL	SR-003, NF-003	PostgreSQL
PostgreSQL 15	Primary data store with constraints and transactions (see 3.6)	F-015, NF-006	-
Media storage	Complaint photos and generated receipts, stored outside the web root and served only through checked views	F-023, F-030	Server file system

3.5 Chosen Architecture Pattern and Rationale

HMS is a layered, modular monolith. It is one deployable Django application with four layers: presentation (templates and optional API), security and cross-cutting services, feature modules (one Django app each), and data access. Each feature module keeps the same internal shape: models, forms, a service layer holding the business rules, views, and URLs. Views never contain business rules; they call the service layer, which is also what the optional API and the tests call.

This pattern was chosen because it keeps the separation of concerns that the SRS asks for (separate Django apps per feature, HMS-NF-006) without the cost of running several services. A four-person team in one semester gains little from network calls between services, and a single database lets allocation and payment updates stay in one transaction. Microservices were rejected as too complex for this scale. A separate JavaScript front end was rejected because the SRS already specifies server-rendered pages.

Key decisions (ADR summary).

ID	Decision	Alternatives rejected	Reason
ADR-01	Layered modular monolith, one Django app per feature	Microservices	Matches team size and timeline; keeps allocation and payment changes in one database transaction; meets HMS-NF-006
ADR-02	Server-rendered Django templates; REST API optional	Single-page app plus API	SRS 2.1 specifies it; less code; CSRF and session protection come built in
ADR-03	PostgreSQL in every environment, including local and CI	SQLite for development	Capacity and unique constraints and row locks must behave the same in tests as in staging (STP 6.1)
ADR-04	Pay on the gateway-hosted checkout; HMS receives only an order reference and a signed webhook	Card form inside HMS	HMS never sees card data, so PCI-DSS exposure is minimal (SR-005, NF-003)
ADR-05	Hide gateway, email/SMS and ID check behind adapters with mock versions	Calling vendor SDKs directly from views	Tests run offline; the gateway can be swapped (Razorpay or Stripe) without touching business code
ADR-06	Lock the room row (select_for_update) and also keep a database CHECK on occupancy	Application check only; optimistic retry	Two admins allocating the last bed cannot both succeed; the database rule is a safety net (HMS-F-015)

ID	Decision	Alternatives rejected	Reason
ADR-07	Append-only audit table; application database role has INSERT and SELECT only on it	Ordinary editable table; log files only	Makes the log immutable in practice (SR-004) and queryable for disputes
ADR-08	Session cookies for the web app; JWT only for an externally exposed API	JWT everywhere	Follows SRS 3.4; sessions are easier to expire after 20 idle minutes and to invalidate on logout

3.6 Technology Stack & Data Stores

Layer	Technology	Why
Language and framework	Python 3.11, Django 5.x	Specified in the SRS; built-in ORM, auth, CSRF and template auto-escaping
Front end	Django templates, Bootstrap 5, small amounts of JavaScript	Responsive from 360 px with little custom CSS
Optional API	Django REST Framework	Standard serializers and permissions; only built if time permits
Database	PostgreSQL 15	Strong constraints, transactions and row-level locks
App server and proxy	Gunicorn behind Nginx	Matches SRS 2.4; Nginx handles TLS and static files
Payments	Razorpay or Stripe, test mode only (final choice open, see 4.6)	PCI-DSS compliant gateway; hosted checkout and signed webhooks
Email/SMS	SMTP or a transactional email API; MailHog in test	OTPs and notifications
PDF and export	A Python PDF library (for example ReportLab or WeasyPrint) and Python csv (final choice open)	Receipts and report export (F-023, F-041)
Packaging and delivery	Docker and Docker Compose, GitHub, Jenkins, SonarQube	Course requires Agile, CI/CD and code review; same containers in every environment

PostgreSQL holds all transactional data. Complaint photos and generated receipts are files on the server, stored outside the web root, with only their paths in the database. The main tables are below. Names are indicative; the final field list is set when the models are written.

Table	Key fields	Rules and constraints
User	id, email, roll_number, password_hash, role (Student, Warden, Admin, Technician), is_active, last_login	Unique email and roll number; password stored only as a salted hash
StudentProfile	user, name, gender, phone, address, guardian and emergency contact	One per student user; gender used for hostel gender-policy checks
OTPToken	user, code_hash, created_at, expires_at, used_at	Expires after 10 minutes; single use; code stored hashed

Table	Key fields	Rules and constraints
Block / Room	Block: name, gender policy. Room: block, floor, number, type (single/shared), capacity, occupancy, is_active	Unique (block, number); CHECK 0 <= occupancy <= capacity
Allocation	student, room, start_date, end_date, status	At most one active allocation per student (partial unique index)
RoomChangeRequest	student, current_room, preferred_room, reason, status, decided_by, decided_at	Status moves Pending to Approved or Rejected only; each change timestamped
FeeStructure	term, room_type, amount, due_date, effective_from	New structures do not change already-paid records
FeeStatement	student, term, amount_due, amount_paid, due_date	Dues = amount_due - amount_paid; never negative
Payment	statement, amount, status (Pending, Success, Failed), gateway_order_id, gateway_txn_id, timestamps	Unique order and transaction ids (stops double credit); no card, CVV or PAN fields exist
Receipt	payment, receipt_no, file_path	One per successful payment
Complaint / ComplaintStatusHistory	ticket_id, student, category, description, photo_path, status, assigned_to; history rows of old and new status with time and actor	Unique ticket id; allowed moves Open to In Progress to Resolved only
AuditLog	actor, action, target_type, target_id, outcome, timestamp, details	Insert-only: no update or delete permission for the application role; database trigger blocks changes

Deployment view:

The same Docker Compose definition builds the local (one per member), CI and staging environments. Staging is a single Linux VM running Nginx with TLS, Gunicorn, PostgreSQL and the gateway sandbox keys. The demo build is a snapshot of the last passing staging build (STP 6.1). Static files are served by Nginx. Gunicorn runs several worker processes, and database connections are reused between requests to help meet the 2-second target.

3.7 Risks & Mitigations

Risk	Likelihood / Impact	Mitigation in the design
Two admins allocate the last bed at the same time	Medium / High	Row lock inside one transaction (ADR-06) plus database CHECK; concurrent-allocation test written in Sprint 2 before the UI (STP risk table)
Payment gateway sandbox is late, throttled or changes	Medium / High	Gateway adapter with a mock version; real sandbox responses saved as test fixtures
Forged or repeated payment webhook credits a student wrongly	Medium / High	Signature check, amount check and a unique transaction id; a replay returns 200 and changes nothing (sequence diagram 4.2.2)
One student can read another student's record by changing an ID in the URL (IDOR)	Medium / High	Every query for student data is filtered by the logged-in user; denied attempts return 403/404 and are logged
Pages become slow with 200 users or 5,000 students	Medium / Medium	Indexed foreign keys, select_related, paginated lists, database-side aggregation for reports; load and scale tests (TC-Perf-01, TC-Scale-01)
Audit table grows large over the 3-year retention period	Medium / Medium	Monthly partitions; older partitions exported to compressed archive files and tested with an archive/restore demo (TC-Ops-01)
Single shared staging VM is busy during load tests	Medium / Medium	Booked slots; fall back to a Dockerised local instance, recording that results are indicative
Optional parts (REST API, Technician role) steal effort from core features	Medium / Medium	Core modules never depend on them; they sit on the same service layer so adding them later needs no redesign
Late SRS change breaks the design	Medium / Medium	Design references are by requirement ID; any change goes through the SRS revision history and the 3.8 table is re-reviewed

3.8 Traceability to Requirements

Every requirement in SRS v1.0 maps to a design reference (DS-*) and a component. These are the same DS-* identifiers used in the design column of the STP RTM (Section 13.2), so the chain requirement, design, module, test case is complete. The last column says where the design is described.

Req ID	Requirement (short)	Design ref	Component	Design summary (section)
HMS-F-001	Register / log in with role	DS-Auth-01	AuthModule	Roll-number check through ID adapter, role on User, session login (4.2.3, 4.3)
HMS-F-002	Student profile update	DS-Auth-03	AuthModule	Student can edit only own StudentProfile (3.6, 4.3)
HMS-F-003	OTP password reset, 10 min	DS-Auth-04	AuthModule	Hashed single-use OTPToken with expiry; throttled (3.6, 4.3)
HMS-F-004	Session timeout, 20 min	DS-Auth-05	Security middleware	Rolling 20-minute session expiry, server-side invalidation (4.2.3)
HMS-F-005	Enforce RBAC	DS-Auth-02	Security middleware	Role check on every view and endpoint; 403 and audit entry on denial (3.9, 4.2.3)
HMS-F-010	View own room and roommates	DS-Alloc-03	AllocationModule	Query filtered to the logged-in student (4.3)

Req ID	Requirement (short)	Design ref	Component	Design summary (section)
HMS-F-011	Admin CRUD on student records	DS-Alloc-04	AllocationModule	Admin-only views; deactivate instead of delete (4.3)
HMS-F-012	Allocate / reallocate room	DS-Alloc-01	AllocationModule	allocate_room service with row lock, gender and availability checks (4.2.1, 4.3)
HMS-F-013	Room-change request	DS-Alloc-05	AllocationModule	RoomChangeRequest state machine; approval calls allocate_room (3.6, 4.3)
HMS-F-014	Manage blocks, rooms, capacity	DS-Alloc-06	AllocationModule	Admin screens on Block and Room; counts update live (3.6)
HMS-F-015	Prevent over-allocation	DS-Alloc-02	AllocationModule, PostgreSQL	Row lock + CHECK constraint; rejected attempt logged (3.5 ADR-06, 4.2.1)
HMS-F-020	Itemised fee statement	DS-Fee-03	FeeModule	Statement = FeeStructure minus Payments (3.6, 4.3)
HMS-F-021	Online fee payment (sandbox)	DS-Fee-01	FeeModule, gateway adapter	Hosted checkout and signed webhook (4.2.2, 4.3)
HMS-F-022	Configure fee structure per term	DS-Fee-04	FeeModule	FeeStructure versioned by effective date (3.6)
HMS-F-023	PDF payment receipt	DS-Fee-05	FeeModule	Receipt generated after payment success; download through checked view (4.2.2, 4.3)
HMS-F-024	Reconcile payments / defaulters	DS-Fee-02	FeeModule	Compare Payments with gateway ledger; defaulters = dues past due date (4.3)
HMS-F-030	Raise maintenance complaint	DS-Maint-01	ComplaintModule	Ticket id, Open status, validated photo upload (3.9, 4.3)
HMS-F-031	Complaint status workflow	DS-Maint-02	ComplaintModule	Allowed-transition check, history rows, notification (4.3)
HMS-F-040	Occupancy / fee reports	DS-Rep-01	ReportingModule	Database-side aggregation for a date range (4.3)
HMS-F-041	CSV and PDF export	DS-Rep-02	ReportingModule	Same report data rendered to CSV or PDF (4.3)
HMS-NF-001	p95 <= 2 s at 200 users	DS-Perf-01	Web UI, all modules	Indexes, query tuning, pagination, Nginx static files (3.6, 4.1)
HMS-NF-002	99.5% availability	DS-Ops-01	Deployment configuration	Process restart policy, uptime monitor, health check (4.4)
HMS-NF-003	Data protection, no card data	DS-Sec-02	All modules	No card fields, per-user data filtering, no personal data in logs (3.9, 4.4)
HMS-NF-004	3-year audit retention	DS-Ops-02	Audit component	Partitioned audit table and archive export (3.7, 4.4)
HMS-NF-005	Responsive and accessible UI	DS-UX-01	Web UI	Bootstrap grid from 360 px, semantic HTML, keyboard access (4.5)
HMS-NF-006	5,000 students / 2,000 rooms; modular	DS-Scale-01	All modules	One Django app per feature; indexed schema (3.5, 3.6)
HMS-SR-001	TLS 1.2+ only	DS-Sec-03	Nginx, Django settings	TLS 1.2+ in Nginx, HTTPS redirect, HSTS, secure cookies (3.9)
HMS-SR-002	Salted password hashing	DS-Sec-01	AuthModule	Django PBKDF2/Argon2 password hashers (3.9)
HMS-SR-003	RBAC at view and API layer	DS-Sec-04	Security middleware, API layer	Same permission classes protect pages and endpoints (3.9, 4.3)
HMS-SR-004	Immutable audit log	DS-Sec-05	Audit component	Insert-only table, trigger, restricted database role (3.5 ADR-07, 3.9)

Req ID	Requirement (short)	Design ref	Component	Design summary (section)
HMS-SR-005	Gateway only, no card data	DS-Sec-06	FeeModule, gateway adapter	Hosted checkout; schema has no card fields (3.5 ADR-04, 4.2.2)

3.9 Security Architecture

Trust boundaries:

There are three - between the user's browser and Nginx (untrusted to trusted, protected by TLS); between HMS and the payment gateway (webhook calls are untrusted until the signature is verified); and between the application and the database (the application uses a limited database role). All input from a browser or webhook is treated as untrusted until checked on the server.

Threat model (STRIDE).

Threat	HMS example	Mitigation
Spoofing	Guessing passwords; replaying a stolen session; fake webhook pretending to be the gateway	Salted password hashes; login and OTP throttling; new session id on login; HttpOnly, Secure, SameSite cookies; webhook signature check
Tampering	Changing the amount in a webhook; editing audit rows; posting forms without a CSRF token	Amount compared with the stored Payment; insert-only audit table; CSRF tokens on every POST; server-side validation of all fields
Repudiation	An admin denies changing a room or fee structure	Audit entry (who, what, when, target, outcome) for every admin action, in the same transaction as the change
Information disclosure	Student reads another student's fee statement or receipt (IDOR); personal data in logs; card data stored	Queries filtered by the logged-in user; 403/404 on mismatch; no sensitive data in logs; no card fields in the schema; TLS 1.2+
Denial of service	Flood of login or OTP requests; very large uploads; heavy report queries	Throttling; upload size limits at Nginx and Django; paginated and indexed queries; Unicorn restart policy
Elevation of privilege	Student calls an Admin URL or API endpoint directly; uploaded script is executed	RBAC enforced on every view and endpoint, not only in the UI; uploads checked by extension, type and size and served as downloads, never executed; least-privilege database role

Security controls mapped to requirements.

Requirement	Design control	Where verified
HMS-SR-001	Nginx accepts TLS 1.2+ only; HTTP gets a 301 to HTTPS; Django SECURE_SSL_REDIRECT, HSTS and secure cookie flags on	TC-Sec-02 (testssl.sh, check --deploy)
HMS-SR-002	Django password hashers (PBKDF2 or Argon2), unique salt per password	TC-Sec-03
HMS-SR-003	One permission layer used by both HTML views and API endpoints; default is deny	TC-Sec-04
HMS-SR-004	Insert-only AuditLog with trigger and restricted role; no edit or delete screens	TC-Sec-05
HMS-SR-005	Hosted checkout; adapter passes only an order reference; schema and logs hold no card data	TC-Sec-06
HMS-NF-003	Per-user data filtering; no personal data or secrets in logs	TC-Sec-01

Additional controls planned for SRS v1.1.

The STP (Section 5.1.2) validates six behaviours that no SRS requirement states yet and proposes them as HMS-SR-006 onwards. The design already supports them:

- Injection and XSS: the ORM builds parameterised queries and templates auto-escape output; raw SQL is not used (STP TC-Sec-07).
- Session and CSRF: CSRF tokens, cookie flags, new session id on login, logout invalidates the session on the server (TC-Sec-08).
- File upload safety: allow-list of extensions and MIME types, size limit, random stored file names, files served without execution (TC-Sec-09).
- Webhook integrity: signature check, amount check, unique transaction id so a replay credits only once (TC-Sec-10).
- Dependency and static analysis: pip-audit, Bandit and SonarQube run as a Jenkins stage; Blocker and Critical findings stop the merge (TC-Sec-11).
- Brute force and OTP abuse: throttling on login and OTP requests; OTP single-use and expires at 10 minutes (TC-Sec-12).

Secrets. Gateway keys, webhook secret, Django SECRET_KEY and database passwords are read from environment variables and never committed to GitHub. Test data is synthetic only.

4. Design

4.1 Design Overview

HMS is designed in layers so that each concern has one home. A request flows like this:

1. Nginx accepts the HTTPS connection and passes the request to Gunicorn.
2. Security middleware loads the session, checks that it has not been idle for 20 minutes, checks the CSRF token on POST requests, and confirms the user's role may use the requested URL.
3. The view reads and validates the form or JSON input and calls a service function in the feature module. Views contain no business rules.
4. The service function applies the rules (for example capacity, gender policy, allowed status changes), changes the data inside a database transaction, and writes an audit entry in the same transaction.
5. After the transaction commits, the service asks the notification adapter to send any email or SMS. A failed message never rolls back a committed change.
6. The view renders a template, or the API returns JSON.

Design rules that apply everywhere.

- Every change that touches more than one table runs inside `transaction.atomic()`.
- Rules that protect data (capacity, unique ids, allowed status values) are enforced both in the service layer and by a database constraint.
- Every query that returns student data is limited to the logged-in student unless the user is Warden/Admin.
- External services are reached only through adapters, so each can be mocked in tests.
- Errors use one common format and error codes (4.4).
- Payments and audit entries are never deleted; a mistake is corrected by a new, linked entry.

Performance and scale. Foreign keys and the columns used in filters (student, room, status, due date) are indexed. List pages are paginated. Views use `select_related` and `prefetch_related` to avoid repeated queries. Reports are computed in the database with aggregates rather than in Python loops. Static files come from Nginx. These choices support 200 concurrent users at p95 of 2 seconds and the 5,000-student, 2,000-room data size; both are measured by TC-Perf-01 and TC-Scale-01.

Feature module design notes.

Module	Main design points
AuthModule (DS-Auth-*)	Registration first asks the ID adapter whether the roll number is valid and unused. Roles are stored on User. Password reset sends a six-digit OTP that is stored hashed, expires in 10 minutes and works once; the request screen gives the same reply whether or not the email exists, so accounts cannot be guessed.

Module	Main design points
AllocationModule (DS-Alloc-*)	The service allocate_room locks the room row, checks that the room is active, that the block's gender policy matches the student, that occupancy is below capacity and that the student has no active allocation, then writes the Allocation and updates occupancy. Reallocation ends the old Allocation and creates a new one in the same transaction. Approving a room-change request calls the same service. Bulk allocation of a batch calls it once per student and reports which students failed and why.
FeeModule (DS-Fee-*)	A statement is built from the FeeStructure for the student's term and room type minus successful Payments. Paying creates a Pending Payment and a gateway order, then redirects to the hosted checkout. Only the verified webhook moves a Payment to Success. Receipts are generated once and stored. Reconciliation lists Payments whose status differs from the gateway ledger and flags students with dues past the due date.
ComplaintModule (DS-Maint-*)	A new complaint gets a unique ticket id and status Open. Photos are checked before saving. Status changes go through one function that allows only Open to In Progress to Resolved, records who changed it and when, and notifies the student.
ReportingModule (DS-Rep-*)	Read-only. Takes a date range, runs aggregate queries over Allocation and Payment, and returns figures that match the underlying records. Export formats share one data builder so the file matches the screen.

4.2 UML Sequence Diagrams

Three flows that are specific to HMS are shown: room allocation (the core rule of the system), online fee payment (the main external integration), and login with role-based access (the base of the security design). Each includes the main alternative path.

4.2.1 Allocate a room (HMS-F-012, HMS-F-015)

The warden submits an allocation. The service locks the room row so that two simultaneous requests for the last bed are handled one after the other. If the room is full, has the wrong gender policy or is inactive, nothing changes, the rejected attempt is logged, and the warden sees a clear message.

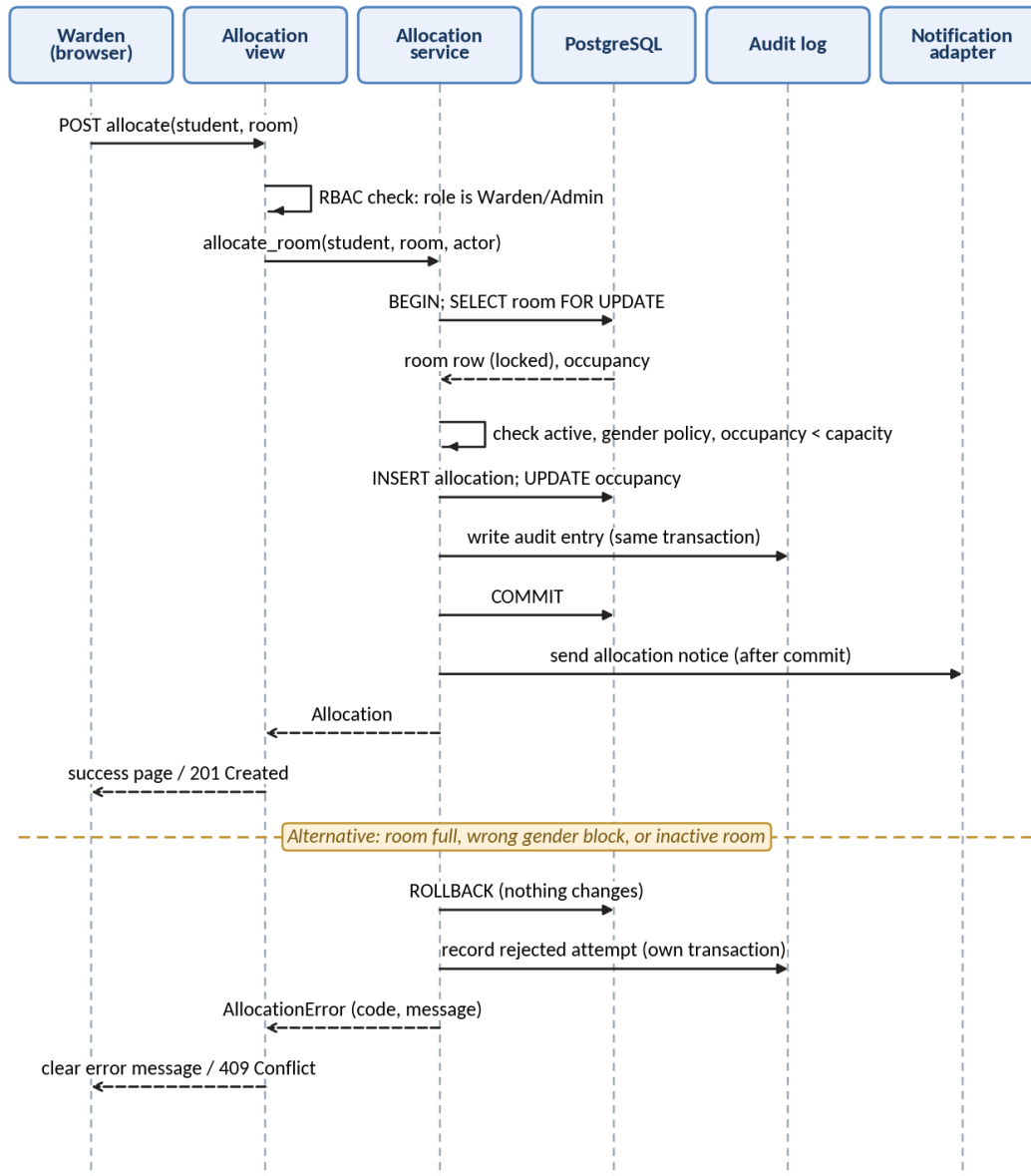


Figure 2: Allocate a room

4.2.2 Pay hostel fees online (HMS-F-021, HMS-F-023, HMS-SR-005)

The student pays on the gateway's own page, so HMS never sees card data. HMS trusts only the signed webhook. A bad signature or a wrong amount is rejected, and a repeated webhook for an already-paid order changes nothing.

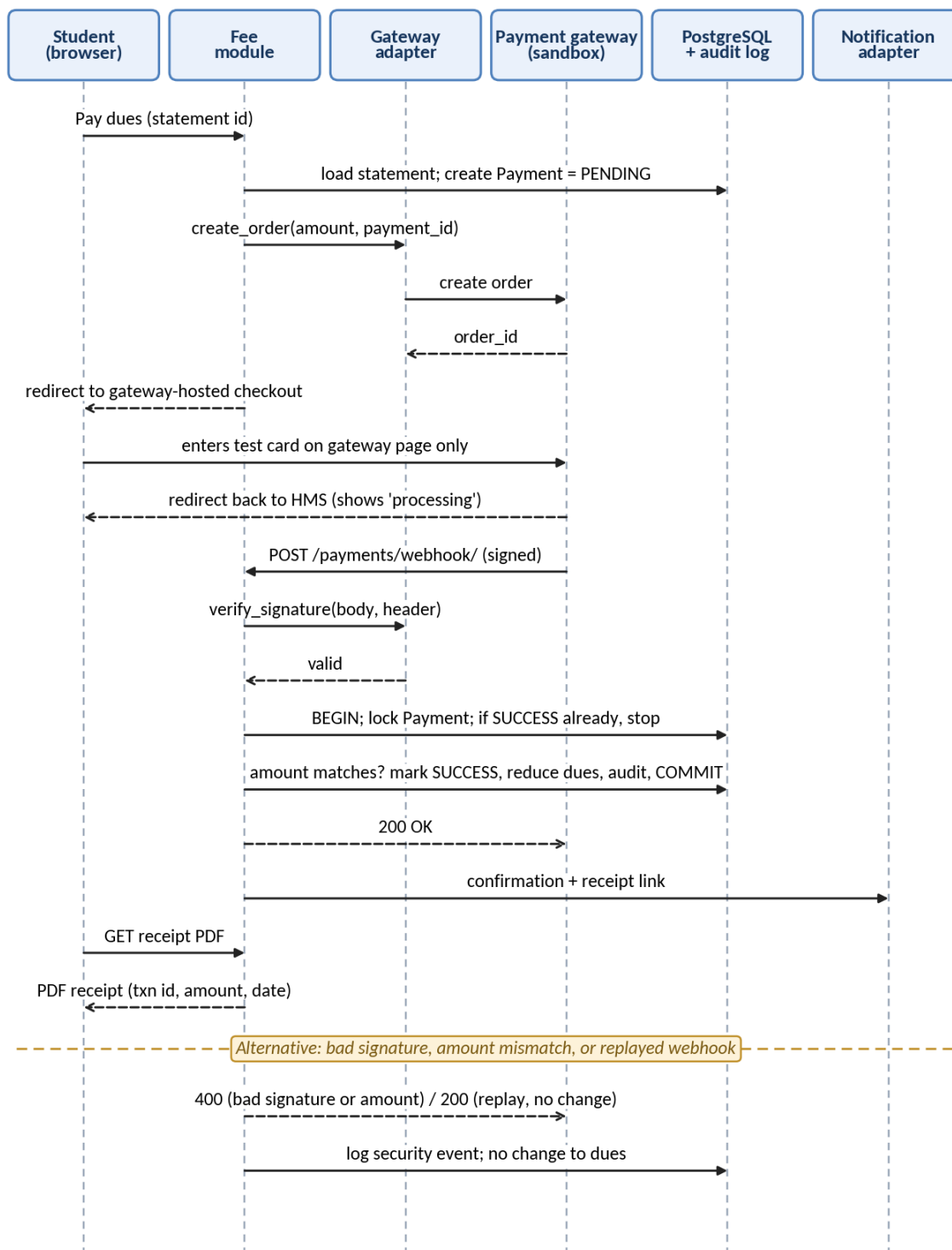


Figure 3: Pay hostel fees online

4.2.3 Log in and enforce access (HMS-F-001, HMS-F-004, HMS-F-005)

After a successful login the user gets a new session. Every later request passes through the access-control middleware, which checks the idle time and the role. A Student who opens an Admin page gets 403 and the attempt is logged; a session idle for 20 minutes is removed and the user is sent back to login.

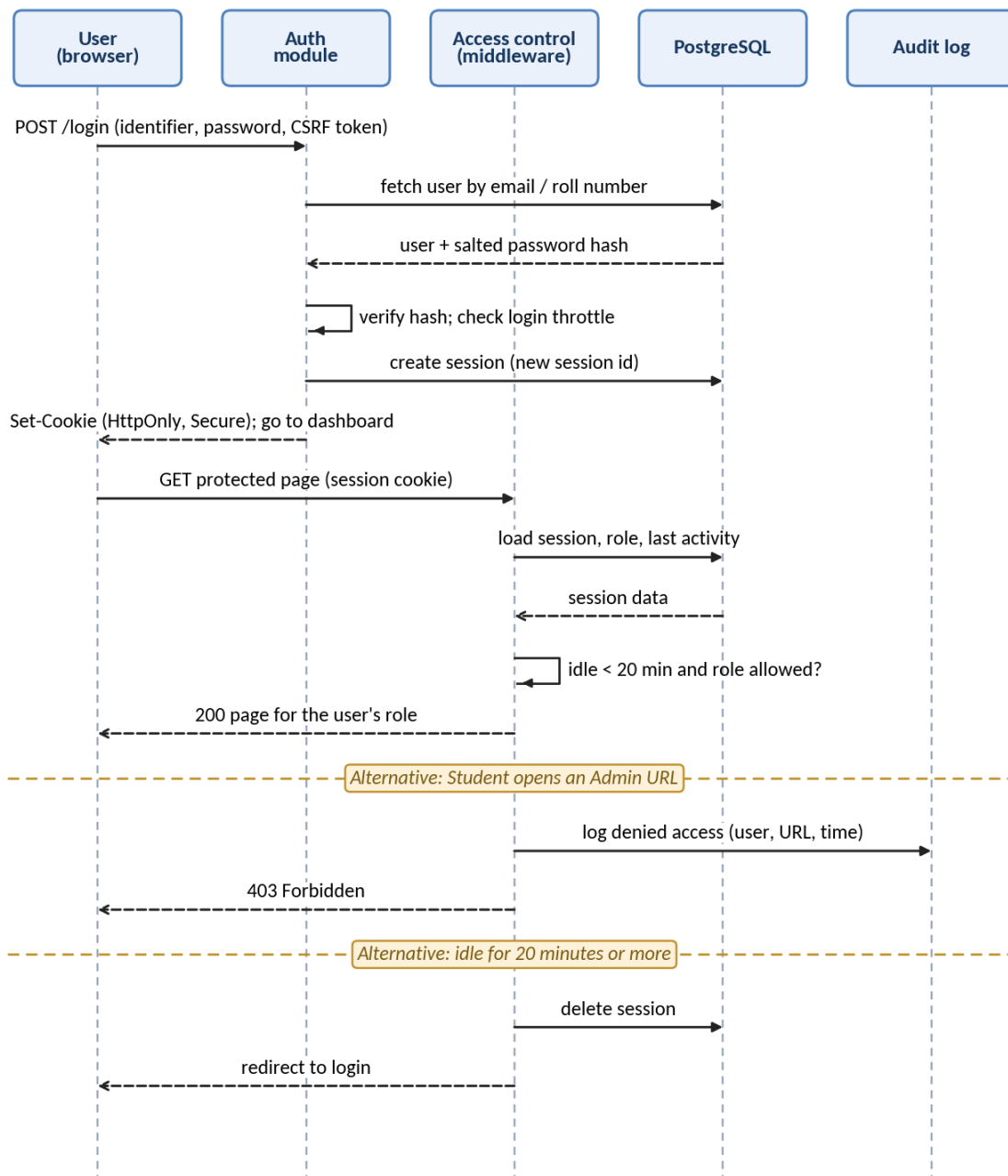


Figure 4: Login and role-based access

4.3 API Design

The same service functions sit behind both the HTML pages and the optional REST API. The tables below define the interfaces of the main components. Paths under `/api/v1/` belong to the optional Django REST Framework layer. The payment webhook is required whether or not the REST API is built. Rules for all endpoints:

- Requests and responses are JSON (complaint creation uses multipart because of the photo). All traffic is HTTPS.
- Web pages use session cookies and CSRF tokens; the API uses the session or, if exposed externally, a JWT in the Authorization header.
- Every endpoint checks the caller's role and, for student data, that the record belongs to the caller. Missing or wrong role returns 403; another student's record returns 403 or 404.
- Errors use the common format in 4.4: {status, code, message, request_id}.

Authentication component (AuthModule)

Method and endpoint	Who	Request	Response	Errors
POST <code>/api/v1/auth/login</code>	Anyone	{identifier, password}	{status, role, idle_timeout_minutes: 20}	400 missing field; 401 invalid credentials; 429 too many attempts
POST <code>/api/v1/auth/register</code>	Anyone	{email, roll_number, password}	{status, user_id, role: "STUDENT"}	400 weak password; 404 roll number not found; 409 already registered
POST <code>/api/v1/auth/password-reset/request</code>	Anyone	{email}	202 {status} (same reply whether or not the email exists)	429 too many requests
POST <code>/api/v1/auth/password-reset/confirm</code>	Anyone	{email, otp, new_password}	{status}	400 OTP invalid, used or older than 10 minutes; 429 too many attempts
PATCH <code>/api/v1/students/me</code>	Student	{phone?, address?, emergency_contact?}	{status, profile}	400 invalid field; 401 not logged in

Allocation component (AllocationModule)

Method and endpoint	Who	Request	Response	Errors
POST <code>/api/v1/allocations/</code>	Warden, Admin	{student_id, room_id}	201 {allocation_id, room: {block, number}, occupancy, capacity}	403 wrong role; 404 student or room not found; 409 ROOM_FULL or STUDENT_ALREADY_ALLOCATED; 422 GENDER_POLICY_MISMATCH
GET <code>/api/v1/allocations/me</code>	Student	-	{room: {block, floor, number}, roommates: [{name}]}	404 no allocation yet
POST <code>/api/v1/room-change-requests/</code>	Student	{preferred_room_id?, reason}	201 {request_id, status: "PENDING"}	409 request already pending
PATCH <code>/api/v1/room-change-requests/{id}</code>	Warden, Admin	{decision: "APPROVED" or "REJECTED", note?}	{request_id, status, decided_at}	404 not found; 409 INVALID_TRANSITION; 409 ROOM_FULL when approving

Fee component (FeeModule)

Method and endpoint	Who	Request	Response	Errors
GET /api/v1/fees/statement	Student	-	{term, items: [...], amount_due, amount_paid, balance, due_date}	404 no fee structure for the student's term
POST /api/v1/payments/	Student	{statement_id}	201 {payment_id, gateway_order_id, checkout_url}	409 NOTHING_DUE; 502 GATEWAY_UNAVAILABLE
POST /payments/webhook/	Gateway only	Headers: signature. Body: {order_id, txn_id, status, amount}	200 {status: "ok"}; a repeated call also returns 200 and changes nothing	400 invalid signature; 400 amount mismatch; 404 unknown order
GET /api/v1/payments/{id}/receipt	Student (owner), Admin	-	application/pdf with transaction id, amount, date, student details	403 not the owner; 404 payment not successful
POST /api/v1/fee-structures/	Admin	{term, room_type, amount, due_date}	201 {structure_id}	400 zero or negative amount; 409 duplicate for term and room type
GET /api/v1/payments/reconciliation	Admin	?from=&to=	{matched, mismatched: [...], defaulters: [...]}	502 GATEWAY_UNAVAILABLE

Complaint component (ComplaintModule)

Method and endpoint	Who	Request	Response	Errors
POST /api/v1/complaints/	Student	multipart: {category, description, photo?}	201 {ticket_id, status: "OPEN"}	400 missing field; 400 INVALID_FILE (type or size); 413 file too large
PATCH /api/v1/complaints/{ticket_id}/status	Admin, Technician (assigned)	{status: "IN_PROGRESS" or "RESOLVED"}	{ticket_id, status, updated_at}	403 not assigned; 404 not found; 409 INVALID_TRANSITION

Reporting component (ReportingModule)

Method and endpoint	Who	Request	Response	Errors
GET /api/v1/reports/{type}	Admin	type = occupancy, vacancy, fee-collection or defaulters. Query: from, to, format = json, csv or pdf	Figures for the range, or a CSV/PDF file with the same data	400 invalid date range; 403 wrong role

Internal interfaces (service layer and adapters)

Interface	Contract	Raises
allocate_room(student, room, actor)	Locks the room, applies all rules, writes Allocation, occupancy and audit entry in one transaction; returns the Allocation	RoomFull, GenderPolicyMismatch, AlreadyAllocated

Interface	Contract	Raises
change_complaint_status(ticket, new_status, actor)	Allows only valid transitions; writes history and audit; triggers notification	InvalidTransition
PaymentGateway.create_order(amount, payment_id)	Returns the gateway order id and checkout URL	GatewayUnavailable
PaymentGateway.verify_signature(body, header)	Returns true only if the signature matches the shared secret	-
PaymentGateway.fetch_ledger(from, to)	Returns gateway transactions for reconciliation	GatewayUnavailable
Notifier.send(user, template, context)	Sends email/SMS; failure is logged and retried, never rolls back business data	-
IdVerifier.check(roll_number)	Returns whether the roll number/ID is valid	VerifierUnavailable
write_audit(actor, action, target, outcome, details)	Inserts one row into AuditLog; called inside the caller's transaction	-

4.4 Error Handling, Logging & Monitoring

Error handling:

All errors use one format so the UI, the API and the tests can rely on it: {status: "error", code, message, request_id}. The message is written for the user and never shows stack traces, SQL or internal paths. Unexpected errors show a generic page and are logged with the same request_id.

Code	HTTP	Message shown to the user
INVALID_CREDENTIALS	401	The email/roll number or password is incorrect.
TOO_MANY_ATTEMPTS	429	Too many attempts. Please wait a few minutes and try again.
OTP_INVALID_OR_EXPIRED	400	This code is incorrect or has expired. Request a new one.
FORBIDDEN	403	You do not have permission to open this page.
ROOM_FULL	409	This room has no free beds. Choose another room.
GENDER_POLICY_MISMATCH	422	This block is not available for this student.
INVALID_TRANSITION	409	This status change is not allowed.
NOTHING_DUE	409	There are no dues to pay for this term.
INVALID_FILE	400	This file type or size is not allowed.
GATEWAY_UNAVAILABLE	502	Payment service is not available right now. Nothing was charged. Please try again later.

Logging:

Three kinds of record are kept, each for a different purpose:

- Application log: structured (JSON) lines with time, level, module, request_id and user id. Rotated daily.
- Audit log (database): who did what to which record, when, and with what outcome, for allocation changes, fee-structure edits, complaint status changes, payments and denied access. Kept for at least 3 years (HMS-NF-004); older monthly partitions are exported to compressed archive files that can be restored on request.

- Access and security log: Nginx access log plus application events for failed logins, throttled requests, denied access (403), bad webhook signatures and rejected uploads.

What is never logged:

Passwords, OTPs, session ids, JWTs, the webhook secret, gateway keys, full request bodies of login or payment calls, and any card data (HMS never receives it). Email addresses and phone numbers appear only in masked form (for example a***@pes.edu). Test and staging data are synthetic.

Monitoring:

An uptime monitor calls a lightweight health-check URL every minute (supports the 99.5% target in HMS-NF-002); Gunicorn and PostgreSQL are set to restart automatically after a crash. The following are watched:

Measure	Alert when	Why it matters
Uptime and health check	Two checks in a row fail	HMS-NF-002
Page response time (p95)	Above 2 seconds over 10 minutes	HMS-NF-001
Server errors (HTTP 5xx) rate	More than 1% of requests	Early sign of a defect or overload
Transaction failure rate (payments Failed or stuck Pending)	Above 5%, or any Pending older than 30 minutes	Money and dues may be out of step
Webhook signature failures	Any burst of failures	Possible forgery or wrong secret
Denied-access (403) and failed-login counts	Sudden rise	Possible probing or brute force
Database connections, disk space, audit table size	Above 80% of limit	Prevents outages and keeps the retention plan realistic

4.5 UX Design

Pages are simple and mobile-first (from 360 px wide), because students mostly use phones during admission windows. Admins get dense but clear tables and bulk actions. Every action shows a clear success or error message. Layouts are checked at 360, 768 and 1366 px (STP, HMS-NF-005). Semantic HTML headings and landmarks, labels on every form field, full keyboard operation with a visible focus mark, colour contrast aimed at WCAG 2.1 AA, text alternatives for images and icons, and error messages that do not rely on colour alone.

Main screens.

Screen	Role	Key content	Requirements
Register / Login / Reset	All	Roll number or email, password, OTP step, clear errors	F-001, F-003
Student dashboard	Student	Room and roommates, fee status and due date, open complaints, link to profile	F-010, F-020, F-030
Room change request	Student	Preferred room, reason, status of earlier requests	F-013
Fee statement and payment	Student	Itemised dues, "Pay now" to the gateway page, receipt downloads	F-020, F-021, F-023
Raise / track complaint	Student	Category, description, optional photo; ticket id and status timeline	F-030, F-031
Admin dashboard	Warden, Admin	Occupancy and vacancy summary, pending requests, defaulters, open complaints	F-040

Screen	Role	Key content	Requirements
Allocation console	Warden, Admin	Student and room search, availability counts, single and batch allocation, error list for failed rows	F-012, F-014, F-015
Student records	Warden, Admin	Create, edit, deactivate	F-011
Fee configuration and reconciliation	Admin	Fee structure per term and room type; reconciliation and defaulter list	F-022, F-024
Reports	Warden, Admin	Date-range filter, tables, CSV and PDF export buttons	F-040, F-041
Complaint queue	Admin, Technician	Assigned complaints, status buttons showing only allowed next steps	F-031

Each user sees only the menu items allowed for their role. Pressing the browser Back button after a payment never resubmits it, because the payment starts from a one-time order.

4.6 Open Issues & Next Steps

Open issue	Proposed next step	Related
Payment gateway not yet chosen (Razorpay or Stripe)	Pick one in Sprint 1 and record it as an ADR; the adapter keeps the rest of the code unchanged	ADR-05, F-021
How the Institute ID check works for the project (live lookup or an imported roll-number list)	Agree with the Course Coordinator; the adapter allows either	F-001
PDF library for receipts and reports not chosen	Short spike in Sprint 2; choose before building F-023	F-023, F-041
Exact gender-policy and preference rules for allocation	Write the decision table (8 combinations, STP 5.4) with the Warden role-player and confirm	F-012
SRS inconsistencies found by the STP: counts of 18 vs 20 functional requirements, "six" feature groups when five are specified, use-case diagrams missing F-003, F-004, F-005, F-015, F-041	Fix in SRS v1.1; this SAD uses 20 functional requirements and five feature groups	STP 13.3
Security behaviours tested in the STP but not yet requirements (injection, CSRF, upload, webhook, dependency scan, throttling)	Add as HMS-SR-006 onwards in SRS v1.1; designs already exist in 3.9	STP TC-Sec-07 to 12
HMS-NF-002 and HMS-NF-004 cannot be fully observed within one semester	Keep as written; verify partially through soak test, restart policy, retention setting and archive/restore demo	STP Section 4
Optional REST API and Technician role	Decide at the Sprint 3 planning meeting; the design supports both without change to core modules	SRS 2.3, 3.3
Retention period for application logs (not set in SRS)	Propose 90 days and confirm with the team	Section 4.4
Safari and iOS cannot be tested with the team's devices	Record as a known limitation in the test summary report	STP Section 4

Next steps:

(1) Review this SAD with the team and the Course Coordinator. (2) Create the Django apps and models from Section 3.6 and add the capacity CHECK and audit trigger in the first migrations. (3) Implement `allocate_room` and its concurrency test first (Sprint 2). (4) Build the gateway adapter with its mock, then the webhook. (5) Update the RTM design column in the SRS with the DS-* references from 3.8.

Future enhancements (outside this project's scope): a mobile app on top of the REST API, automatic fee reminders by SMS, and integration with the university ERP beyond the ID check.

5. Appendices

5.1 Glossary

Term	Plain-language meaning
ADR	Architecture Decision Record: a short note saying what was decided, what else was considered, and why.
Adapter	A small piece of code that hides an outside service (payment, email, ID check) behind a simple interface so it can be swapped or mocked.
Audit log	A permanent record of who did what and when, used to trace actions and settle disputes.
CSRF	Cross-Site Request Forgery: a trick where another site makes your browser send an unwanted request. Blocked with a secret token on every form.
DRF	Django REST Framework: add-on library for building JSON APIs in Django.
DS-* reference	A design identifier (for example DS-Alloc-01) that links a requirement to the part of the design that satisfies it.
HSTS	A header telling browsers to always use HTTPS for the site.
Idempotent	Safe to repeat: doing it twice has the same effect as doing it once (used for webhooks).
IDOR	Insecure Direct Object Reference: reaching someone else's record by changing an id in the address.
JWT	JSON Web Token: a signed token used to prove identity to an API.
ORM	Object-Relational Mapping: lets code work with database tables as Python objects.
OTP	One-Time Password: a short code that works once and expires quickly.
p95	The response time that 95% of requests are faster than.
PAN	Primary Account Number: the long number on a payment card.
PBKDF2 / Argon2	Slow, salted password-hashing methods that make stolen password databases hard to crack.
PCI-DSS	Payment Card Industry Data Security Standard: rules for handling card data.
RBAC	Role-Based Access Control: what a user may do depends on their role (Student, Warden/Admin, Technician).
STRIDE	A checklist of six threat types: Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege.
TLS	Transport Layer Security: encryption of data between browser and server (HTTPS).
Webhook	A message the payment gateway sends to HMS to say a payment finished.
WCAG	Web Content Accessibility Guidelines: rules for making web pages usable by everyone.

5.2 References

- ISO/IEC/IEEE 42010: Systems and software engineering - Architecture description.
- IEEE 829-2008: Standard for Software and System Test Documentation (used by the STP).
- OWASP Top 10 and OWASP Application Security Verification Standard (ASVS).
- NIST SP 800-160: Systems Security Engineering.
- PCI DSS (Payment Card Industry Data Security Standard) and the chosen gateway's sandbox and webhook documentation.

- Django documentation: security, transactions and select_for_update, sessions, password hashing.
- WCAG 2.1 (W3C).
- HMS Software Requirements Specification v1.0, 04-09-2026.
- HMS Software Test Plan v1.0, 16-09-2026.
- Jackfruit - Software Engineering Project Guidelines, UE24CS341A, 2026.

5.3 Tools

Purpose	Tool
Diagrams in this document	Graphviz (component diagram) and Python/Matplotlib (sequence diagrams). draw.io or PlantUML can be used to redraw them for later editing.
API documentation	Swagger/OpenAPI (generated from DRF if the API is built); Postman collections for testing
Build, test and quality	Docker Compose, GitHub, Jenkins, pytest, Selenium, Locust, SonarQube, Bandit, pip-audit (as in the STP)
Security checks	OWASP ZAP, testssl.sh, Django check --deploy
Mail capture in test	MailHog